Міністерство освіти і науки України
Харківський Національний Університет Радіоелектроніки
Кафедра ПІ

Звіт

Практична робота №1

з дисципліни ”Методологія конструктивного мислення для наукових досліджень”

Виконав:
ст. гр. ІПЗм-26-3
Костюк Деніс Васильович

Перевірив:
доц. каф. ПІ
Каук Віктор Іванович

Харків 2026

ЗМІСТ

1 Хід роботи 3
1.1 Дослідницьке поле 3
1.2 Ключові проблеми 3
1.2.1 Методологія 3
1.2.2 Знайдені ключові джерела 4
1.2.3 Виявлені проблеми 5
1.3 Перевірені дослідницькі прогалини 6
1.4 Формування тем 8
1.4.1 Тема A (аналітична-теоритична) 8
1.4.2 Тема B (інженерна) 13
1.4.3 Тема C (міждисплінарна) 18
1.5 Порівняння тем 25
1.6 Peer review 27
1.6.1 Відгуки щодо теми A 27
1.6.2 Відгуки щодо теми B 27
1.6.3 Відгуки щодо теми C 27
1.7 Decision Matrix 27
Висновок 29

1 ХІД РОБОТИ

1.1 Дослідницьке поле

Дослідницьке поле роботи – дослідження продуктивності та надійності диспетчеризації віртуальних потоків на платформі JVM в умовах взаємодії з нативним кодом (FFM та JNI) і блокуючими операціями вводу-виводу. Англійською: Performance and scheduling reliability analysis of JVM virtual threads under native boundary interactions (FFM and JNI) and blocking I/O workloads.

Вибір поля обумовлено особистим інтересом до внутрішньої архітектури віртуальних машин, розробки високонавантажених серверних систем на Java/Kotlin та актуальністю переходу на віртуальні потоки (Project Loom) разом із новим інтерфейсом взаємодії з нативною пам'яттю (Panama FFM API).

Для обрання дослідницького поля виконувався пошук за темами, що відповідали моєму технічному стеку та сфері наукових інтересів (Java/Kotlin, багатопоточність, рантайми, системна оптимізація), по таких наукових репозиторіях і базах даних, як ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, arXiv, Google Scholar, а також інженерних репозиторіях OpenJDK (JEPs). Джерела, використовувані надалі, було також знайдено через них.

1.2 Ключові проблеми

1.2.1 Методологія

Для формування бібліографічної бази та аналізу предметної області використовувався комбінований підхід: пошукова система Perplexity для первинної навігації та модульний булевий (бінарний) пошук у наукометричних базах (ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, arXiv, Google Scholar). Перелік пошукових запитів:

- Базова поведінка диспетчера та масштабованість:
  - ”Virtual Threads” AND ”carrier starvation”
  - ”Project Loom” AND ”tail latency”
  - ”Virtual Threads” AND ”scalability”
- Нативний пінінг та міграція (JNI / FFM API):
  - ”Virtual Threads” AND ”carrier pinning” AND ”JNI”
  - ”Virtual Threads” AND ”FFM” AND ”pinning”
  - ”Foreign Function Memory” AND ”JNI” AND ”benchmark”

- Еволюція моніторів синхронізації (JEP 491 / JDK 24+):

– ”Virtual Threads” AND ”JEP 491”
– ”Virtual Threads” AND ”synchronized” AND ”pinning”
– ”residual pinning” AND ”JDK 24”

- Міжрівнева телеметрія та моделювання:

– ”Virtual Threads” AND ”eBPF” AND ”scheduler”
– ”Virtual Threads” AND ”queueing model”
– ”Virtual Threads” AND ”JFR” AND ”pinning”

1.2.2 Знайдені ключові джерела

1. FCoral: A Framework for Fine-grained Concurrency Evaluation Across Programming Language Runtimes. 2026. (https://doi.org/10.1145/3847668).
2. Misleading Microbenchmarks on the Java Virtual Machines. 2026. (https://doi.org/10.1145/3748522.3779882).
3. Practical Benchmarking Configurations for Reproducible Execution-Time Measurements of CI/CD-Style Workloads. 2026. (https://doi.org/10.1145/3838807).
4. Experimental Evaluation Methodology for the Era of No Steady Performance. 2026. (https://doi.org/10.1145/3839448).
5. Fray: An Efficient General-Purpose Concurrency Testing Platform for the JVM. 2025. (https://doi.org/10.1145/3764119).
6. Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads. 2024. (https://norma.ncirl.ie/8134/).
7. Virtual Threads after JDK 24: What Changed for Production Java. 2026. (https://www.infoq.com/articles/virtual-threads-after-jdk24/).
8. JEP 491: Synchronize Virtual Threads without Pinning. 2024. (https://openjdk.org/jeps/491).
9. Java Threading Options in 2026: Platform vs Virtual Threads. 2026. (https://www.jobrunr.io/en/blog/java-job-threading-options/).
10. Java 25 and the Evolution of Modern Java for Next-Generation Software Systems. 2026. (https://doi.org/10.55041/ijcope.v2i4.357).

11. eBPF jAgent: Transaction-level Resource Demand Tracing for Java Applications Using eBPF and OpenTelemetry. 2026. (`https://github.com/hm-green-it-lab/ssp2026`).

1.2.3 Виявлені проблеми

На основі аналізу наукової літератури, офіційних специфікацій OpenJDK та результатів інженерних досліджень виділено ключові проблеми диспетчеризації та надійності віртуальних потоків JVM:

1. **Залишкове блокування пулу потоків-носіїв на нативній межі (carrier-pool starvation).** Незважаючи на усунення моніторного пінінгу в JDK 24 (JEP 491), виконання нативного коду через C ABI (як у JNI, так і в Panama FFM API) та нативні зворотні виклики (callbacks) примусово утримує платформені потоки, спричиняючи приховане виснаження пулу ForkJoinPool при зростанні конкурентності.

2. **Відсутність замкнених аналітичних моделей масового обслуговування.** У науковій літературі бракує формалізованого математичного апарату теорії черг, який би дозволяв аналітично розраховувати критичний поріг насичення системи (λ_{crit}), довжину черг очікування та квантилі хвостових затримок для гібридних навантажень, що поєднують відмонтовувані операції вводу-виводу та невідмонтовувані нативні виклики.

3. **Ефект взаємного блокування задач та радіус ураження (blast radius).** Неізольоване закріплення потоків-носіїв під час тривалих нативних операцій блокує спільний пул ресурсів, що призводить до нелінійного падіння пропускної здатності та спалахів хвостових затримок (P_{99}) у сусідніх незалежних віртуальних потоках (зокрема неблокуючому мережевому I/O).

4. **Зміщення вузького місця та дефіцит архітектурних патернів ізоляції.** Неконтрольоване масштабування віртуальних потоків зміщує перевантаження на супутні підсистеми (пули з'єднань з базами даних, системні дескриптори), зумовлюючи каскадні відмови; водночас в інженерній практиці бракує верифікованих патернів захисту (семафорних бар'єрів-bulkheads, виділених пулів) для ізоляції нативного коду.

5. **Міжрівневий розрив спостережуваності рантайму та ядра ОС.** Планувальник JVM керує легковажними потоками у просторі користувача (M :

N), а операційна система – на рівні ядра (task_struct). Профайлери байткоду (JVMTI) не фіксують перемикання контексту ядра ОС (sched_switch, черги futex), тоді як системні трейсери (eBPF) прив'язані до потоків ядра та втрачають транзакційний контекст при виконанні віртуальних потоків.

6. **Ефект спостерігача та висока вартість багатофакторного тестування.** Глибоке системне трасування вносить значний оверхед у рантайм і спотворює вимірювання затримок, а повний емпіричний аналіз багатовимірного простору конфігурацій потребує надмірних ресурсів, що зумовлює потребу у легковажній телеметрії та сурогатних моделях машинного навчання.

1.3 Перевірені дослідницькі прогалини

Результати верифікації виявлених дослідницьких прогалин за правилом тріангуляції наведено в табл. 1.1.

Таблиця 1.1 – Верифікація дослідницьких прогалин та їх доказова база

Ідентифікатор	Формування	Опис та доведення
GEC-1, Енергія VT при пінінгу	На тлі декларованої енергоефективності віртуальних потоків на ізольованих задачах залишається недослідженим їхнє питоме енергоспоживання під час деградації супутніх пулів ресурсів та нативного пінінгу.	Неможливо довести за допомогою правила тріангуляції (у наявній доказовій базі менше 3 джерел із DOI; промислові звіти [Src-10], [Src-12] та специфікація [Src-11] не мають DOI). Підтверджуючі джерела з DOI: [Src-9] (DOI: `https://doi.org/10.1016/j.jss.2026.112963`) фіксує енергозбереження лише для чистих REST-профілів; [Src-1] (DOI: `https://doi.org/10.1145/3847668`) вимірює затримки віртуальних потоків без оцінки енергоспоживання.

Продовження таблиці 1.1

Ідентифікатор	Формування	Опис та доведення
GEC-2, Джиттер і планування на big.LITTLE	Теоретичний аналіз поширення часового джитеру в ланцюжках періодичних завдань та практичні алгоритми енергоефективного диспетчеризування на процесорах ARM big.LITTLE розвиваються ізольовано.	Доведено за правилом триангуляції за трьома незалежними рецензованими джерелами з DOI: 1) моделі поширення джитеру конвеєрів абстраговані від планувальника ОС та енергії ([Src-15], DOI: https://doi.org/10.1109/RTSS66672.2025.00017); 2) алгоритми планування на big.LITTLE оптимізують дедлайни без контролю фазового джитеру ([Src-17], DOI: https://doi.org/10.1109/JIOT.2025.3568691; [Src-18], DOI: https://doi.org/10.1145/3716368.3735195); 3) системні огляди підтверджують метричний розрив між дедлайнами та енергією ([Src-16], DOI: https://doi.org/10.1016/j.sysarc.2025.103628).

Продовження таблиці 1.1

Ідентифікатор	Формування	Опис та доведення
GEC-3, Нативний бар'єр FFM/JNI у VT	Незважаючи на усунення моніторного пінінгу в JDK 24, поведінка нативного бар'єру FFM API у порівнянні з JNI та вводом-виводом залишається експериментально нерозчленованою в умовах насичення пулу носіїв.	Доведено за правилом триангуляції за трьома незалежними рецензованими джерелами з DOI: 1) наявні бенчмарки тестують рантайм як «чорну скриньку» або маскують деградацію пулу сокетами ([Src-1], DOI: https://doi.org/10.1145/3847668; [Src-8], DOI: https://doi.org/10.7148/2025-0555); 2) доведено нерепрезентативність ізольованих мікротестів нативного переходу через спотворення JIT-профілів ([Src-2], DOI: https://doi.org/10.1145/3748522.3779882); 3) доведено нестаціонарність фаз JVM, що спростовує наївні середні оцінки деградації пулу носіїв ([Src-4], DOI: https://doi.org/10.1145/3839448).

1.4 Формування тем

Для подальшого формування тем було обрано **GEC-3**. Вибір проводився за допомогою виключення. **GEC-1 –** відпала, бо недоводиться за правилом триангуляції. **GEC-2** непідходить через те що наявне устаткування SoC qualcom 425 – має архітектуру 4x Cortex-A53, яка є виключно енергоефктивною (LITTLE). Тому почнемо формувати тему навколо **GEC-3**.

1.4.1 Тема A (аналітична-теоритична)

Тема A є теоретико-аналітичним кандидатом дослідження, спрямованим на математичне моделювання процесів диспетчеризації віртуальних потоків JVM.

Вона поєднує методи теорії масового обслуговування із системними механізмами ядра віртуальної машини HotSpot для розрахунку затримок та оцінки навантаження пулу потоків-носіїв при взаємодії через FFM та JNI.

Заголовок (UA): Аналітичне моделювання та експериментальна верифікація затримок віртуальних потоків на межі взаємодії FFM та JNI у різних версіях JVM.

Заголовок (EN): Analytical Modeling and Experimental Verification of Virtual Thread Delays Across the FFM and JNI Boundary Under Stratified JVM Versions.

Дослідницьке поле: Комп'ютерні науки (спеціальність 122) та інженерія програмного забезпечення (спеціальність 121) – математичне моделювання обчислювальних систем, теорія черг, аналіз продуктивності системного програмного забезпечення.

Наукова проблема: Перехід від JNI до Panama FFM змінює спосіб взаємодії Java з нативним кодом, проте нативний бар'єр залишається джерелом блокування потоків-носіїв (carrier pinning) у моделі віртуальних потоків ($M : N$). У той час як стандартні операції (сокетний I/O) легко відмонтовуються і звільняють ресурси, нативні C-виклики примусово утримують обмежені потоки процесора. Це призводить до різкого зростання черг і хвостових затримок, математичний зв'язок яких із кількістю потоків-носіїв, інтенсивністю навантаження та тривалістю нативних операцій наразі не охарактеризовано.

Доказова база проблеми:

1. Специфікація OpenJDK JEP 491 ([Src-11]): у JDK 24 усунуто моніторний пінінг (`synchronized`), проте нативні виклики C ABI (у JNI та Panama FFM) та нативні зворотні виклики (callbacks) залишаються нерозв'язаним джерелом блокування потоків-носіїв.

2. Емпіричні межі віртуальних потоків Pandita ([Src-7]): віртуальні потоки в JDK 21 дають приріст продуктивності лише для чистого введення-виведення, тоді як навантаження з блокуванням процесорних ресурсів не отримують переваг.

3. Хвостова деградація у фреймворку FCoral ([Src-1], DOI: `https://doi.org/10.1145/3847668`): зафіксовано спалахи затримок P_{99} віртуальних потоків при зростанні конкурентності, проте рантайм оцінювався як «чорна скринька» без ізоляції нативних викликів.

4. Нестаціонарність рантайму JVM у Antoch et al. ([Src-4], DOI: `https:`

//doi.org/10.1145/3839448): робочі навантаження JVM характеризуються фазовими перепадами через оптимізації JIT і паузи збирача сміття, що вимагає врахування розкиду вимірювань під час верифікації моделей черг.

Дослідницька прогалина: Класичні математичні моделі черг не підходять для віртуальних потоків Java: вони розглядають потоки або як повністю легковажні (що завжди звільняють процесор), або як важкі потоки ОС. У реальності виникає змішане навантаження: звичайні веб-запити легко вивільняють ядра процесора, а нативний C-код (JNI чи новий FFM API) примусово утримує їх, викликаючи раптовий колапс системи.

• **Математична модель змішаної черги:** створення моделі розрахунку черг і затримок для гібридного навантаження, де частина потоків вільно відмонтовується, а частина – блокує ядра процесора.

• **Визначення точки перевантаження:** аналітична оцінка критичного порогу навантаження, за якого пул потоків-носіїв вичерпується, а затримки (P_{99}) різко зростають.

• **Математичне порівняння технологій:** з'ясування, чи змінює заміна JNI на FFM та оновлення версії Java (JDK 21 проти JDK 25) межі насичення системи.
Тип прогалини: Theoretical gap.

Об'єкт дослідження: Процес планування, відмонтування та закріплення потоків-носіїв віртуальних потоків при одночасному виконанні нативних (JNI/FFM) та керованих операцій у віртуальній машині Java.

Предмет дослідження: Моделі теорії масового обслуговування змішаних черг та емпіричні розподіли затримок планувальника `ForkJoinPool` на межі взаємодії JNI, FFM та I/O за різного паралелізму носіїв у версіях JDK 21 та JDK 25.

Дослідницькі питання (RQ):

• **RQ1:** Наскільки точно модель масового обслуговування здатна апроксимувати затримки в черзі, коли віртуальні потоки поєднують відмонтовувані операції (сокети) та нативні виклики (JNI/FFM)?

• **RQ2:** За якого співвідношення інтенсивності навантаження, кількості носіїв та тривалості нативних викликів настає вичерпання пулу планувальника?

• **RQ3:** Як перехід від JNI до FFM та оновлення версії JVM (JDK 21 проти JDK 25) впливають на теоретичні межі стійкості черги?

Наукова гіпотеза: Час перебування віртуальних потоків у черзі спільного

пулу носіїв підпорядковується закономірностям системи масового обслуговування зі змішаними класами вимог. Існує поріг інтенсивності нативного навантаження відносно кількості потоків-носіїв, перевищення якого призводить до різкого нелінійного зростання хвостових затримок (P_{99}). При цьому FFM забезпечує менший оверхед на коротких нативних операціях, але за тривалого блокування або наявності зворотних викликів (callbacks) межі вичерпання пулу для FFM і JNI стають еквівалентними незалежно від оптимізацій моніторів у JDK 24+.

Мета дослідження: Побудувати та експериментально перевірити розрахункову модель черги віртуальних потоків, яка дозволяє оцінювати затримки та прогнозувати поріг перевантаження потоків-носіїв при викликах нативного коду через JNI та FFM у різних версіях JVM.

Основні завдання:

1. Формалізувати стани переходів планувальника віртуальних потоків JVM (`ForkJoinPool`) за наявності відмонтовуваних операцій та нативного пінінгу.

2. Сформулювати модель масового обслуговування для оцінки довжини черги та затримок при змішаному навантаженні.

3. Спроєктувати та реалізувати факторіальний стенд для вимірювання затримок JNI та FFM при різній тривалості блокування, конкурентності та кількості носіїв на JDK 21 і JDK 25.

4. Провести експериментальні прогони, зіставити емпіричні затримки з розрахунковими значеннями та оцінити межі застосовності моделі.

Наукова новизна: Запропоновано розрахункову модель черги віртуальних потоків, яка, на відміну від наявних підходів, враховує одночасну наявність кооперативних операцій та нерозвантажуваного нативного блокування (JNI/FFM) у середовищах JDK 21 та JDK 25.

Практичне значення: Методика оцінки параметрів черги для розробників, що дозволяє заздалегідь оцінювати допустиму інтенсивність нативних операцій відносно кількості процесорних ядер без необхідності проведення тривалого емпіричного тестування кожної конфігурації.

Методологія та методи дослідження: Теорія масового обслуговування (моделі черг із різними типами вимог), чисельні розрахунки та обробка даних у Python (NumPy, SciPy), лабораторний експеримент із мікробенчмаркінгом на базі JMH за методикою усунення JIT-викривлень ([Src-2]), а також телеметрія HotSpot

JVM через Java Flight Recorder (JFR) та HdrHistogram.

Базовий рівень (Baseline) та вибірка даних: Базовий рівень – віртуальні потоки OpenJDK 21 LTS із дефолтним числом носіїв, що виконують виключно чисто керовані блокування (`LockSupport.parkNanos` / сокети) без навантаження нативним кодом. Вибірка даних – емпіричні ряди затримок (HdrHistogram) та подій блокування (JFR), зібрані на локальному стенді для контрольованих конфігурацій із повторними прогонами для статистичної достовірності.

Метрики оцінки: Розрахункові: середня похибка апроксимації (MAPE) та коефіцієнт відповідності (R^2) між розрахунковими та емпіричними значеннями. Емпіричні: пропускна здатність (ops/sec), хвостові затримки (P_{50}, P_{95}, P_{99}), тривалість утримання потоку-носія.

Інструменти та апаратне забезпечення: OpenJDK 21 LTS та 25 LTS, GCC/Clang, Make, JMH, JFR, HdrHistogram, Python 3 (NumPy, SciPy, Matplotlib). Апаратна платформа: Intel Core i5-10400F (6 ядер / 12 потоків), 32 GB RAM, ОС Linux із фіксацією ядер через `taskset`.

Загрози валідності: Внутрішня – спотворення затримок через JIT-прогрів і паузи GC (мінімізація: warmup-ітерації, фіксований heap, статистична сегментація вимірів за [Src-4]); конструктивна – викривлення ізольованих JMH-тестів (мінімізація: тестування на змішаному навантаженні за [Src-2]); зовнішня – специфіка тестування на одній архітектурі x86_64 без урахування NUMA-факторів великих серверів.

Мінімальне життєздатне дослідження (MVR) та Stretch Goal: MVR – базова модель черги для двох типів задач (відмонтовувані та нативні), перевірена на JDK 21 та JDK 25 для JNI та FFM при 3 рівнях паралелізму носіїв та різній тривалості блокування. Stretch Goal – розширення моделі для врахування додаткових зворотних викликів (callbacks) із нативного коду в Java під час змішаного навантаження.

Доказова база прогалини: Аналітичний огляд [SGP-3] та картка [GEC-3] фіксують теоретичний розрив між класичними моделями систем масового обслуговування та реальною динамікою $M : N$ планувальників: наявні роботи або оцінюють рантайм емпірично як «чорну скриньку» ([Src-1]), або аналізують ізольовані мікротести без формалізації черг носіїв ([Src-2], [Src-4]), залишаючи

взаємодію нативних і кооперативних класів задач без розрахункового апарату.

Критерій фальсифікованості: Публікація верифікованої моделі черг для FFM та JNI під контрольованим паралелізмом носіїв або експериментальне доведення того, що затримки черги `ForkJoinPool` мають суто стохастичний недетермінований характер через шум ОС, який не піддається математичному моделюванню.

Обґрунтування вибору та ризики теми: Тема забезпечує теоретичний фундамент (спеціальність 122 «Комп'ютерні науки»), поєднуючи класичну прикладну теорію черг із актуальними змінами рантайму JVM (Loom та Panama), що гарантує наукову новизну та публікабельність. Головний ризик полягає у математичній складності: рівняння черг для неекспоненційних розподілів часу нативних операцій можуть виявитися надто громіздкими, змушуючи спиратися на наближені чисельні методи замість простих формул.

1.4.2 Тема B (інженерна)

Тема B є прикладним інженерним кандидатом дослідження, орієнтованим на вирішення практичних проблем стійкості (resilience) та продуктивності високонавантажених бекенд-систем на базі JVM. Вона зосереджена на емпіричному дослідженні взаємного впливу нативних викликів FFM/JNI та операцій вводу-виводу на спільний пул потоків-носіїв віртуальних потоків, а також на проєктуванні та експериментальній верифікації практичних патернів ізоляції ресурсів (bulkheading).

Заголовок (UA): Експериментальне дослідження продуктивності та розробка патернів ізоляції пулу носіїв віртуальних потоків при взаємодії через FFM та JNI під контрольованим навантаженням.

Заголовок (EN): Empirical Performance Evaluation and Carrier-Pool Isolation Patterns for Virtual Threads Across the FFM and JNI Boundary Under Controlled Pressure.

Дослідницьке поле: Інженерія програмного забезпечення (спеціальність 121), архітектура програмних систем, високопродуктивні обчислення на платформі JVM, патерни надійності та стійкості (Resilience & Bulkheading).

Наукова проблема: При переведенні високонавантажених сервісів на віртуальні потоки та заміні традиційного JNI на Panama FFM API виникає проблема

неконтрольованого закріплення потоків-носіїв (carrier pinning). Нативні виклики та зворотні виклики (callbacks) блокують спільні потоки процесора, що призводить до деградації сусідніх операцій вводу-виводу (сокетних і файлових), які виконуються на тому ж пулі носіїв. При цьому відсутні систематичні емпіричні дані щодо меж стійкості такої системи та ефективності прикладних патернів ізоляції ресурсів.

Доказова база проблеми:

1. Зниження продуктивності у звітах JobRunr ([Src-12]): неконтрольований пінінг потоків-носіїв спричиняв падіння пропускної здатності у 14 разів (з 462 до 33 jobs/s) та зростання затримок P_{99} з 253 мс до 6799 мс (у 27 разів).

2. Зміщення вузького місця у дослідженнях InfoQ ([Src-10]): масштабування віртуальних потоків без обмеження конкурентності перевантажує супутні пули ресурсів (зокрема пул з'єднань БД HikariCP, спричиняючи близько 37% помилок) через відсутність перегородок (bulkheads); також показано, що після усунення моніторного пінінгу в JDK 24 нативні виклики (JNI/FFM) залишаються головним джерелом блокування носіїв.

3. Офіційні застереження Quarkus та OpenJDK ([Src-11]): нативні зворотні виклики (callbacks) із C у Java блокують потік-носій незалежно від версії JVM.

4. Емпіричний огляд платформи Java 25 ([Src-24]): накладні витрати виклику FFM API наблизилися до JNI (відхилення в межах 1.8% для пакетних задач), що вказує на те, що головним фактором деградації є не вартість переходу в нативний код, а утримання потоку-носія під час тривалого блокування.

Дослідницька прогалина: Віртуальні потоки ефективно масштабують операції введення-виведення, проте нативний код (через JNI або FFM) блокує спільні потоки процесора і перешкоджає виконанню сусідніх завдань. Спроби компенсувати це збільшенням пулу носіїв призводять до значного зростання споживання оперативної пам'яті.

- **Межа стійкості планувальника:** за якого співвідношення інтенсивності вхідного навантаження, кількості потоків-носіїв та тривалості нативних операцій відбувається перехід від стабільної роботи до різкого зростання хвостових затримок (P_{99}) для JNI та Panama FFM.

- **Ефективність патернів ізоляції:** наскільки результативно прикладні механізми ізоляції (обмеження паралелізму через семафори за патерном Bulkhead або винесення нативних викликів у виділені пули) запобігають вичерпанню

`ForkJoinPool` при помірних накладних витратах на пам'ять і затримку.
Тип прогалини: Empirical gap.

Об'єкт дослідження: Високонавантажені програмні системи на базі JVM, що одночасно виконують нативні (JNI/FFM) операції та мережеві й файлові операції введення-виведення у віртуальних потоках.

Предмет дослідження: Емпіричні показники пропускної здатності, квантилі затримок (P_{95}, P_{99}), а також архітектурні патерни ізоляції ресурсів (семафорні перегородки Bulkhead, виділені пули платформних потоків для нативного коду, налаштування паралелізму планувальника).

Дослідницькі питання (RQ):

• **RQ1:** За однакової семантики операцій, як співвідносяться JNI та FFM за пропускною здатністю, накладними витратами процесора та хвостовими затримками при зміні тривалості нативного блокування, конкурентності та наявності зворотних викликів (callbacks)?

• **RQ2:** Наскільки сильно закріплені нативні виклики JNI та FFM збільшують затримки сусідніх сокетних операцій на спільному пулі носіїв у різних версіях JVM (JDK 21 проти JDK 25)?

• **RQ3:** Чи дозволяють прикладні механізми ізоляції (семафорні бар'єри або пули розвантаження перед нативним викликом) запобігти вичерпанню пулу носіїв із меншими витратами оперативної пам'яті, ніж збільшення розміру пулу планувальника?

Наукова гіпотеза: Panama FFM забезпечує нижчі накладні витрати виклику порівняно з JNI для коротких операцій, проте за тривалого блокування або наявності зворотних викликів (callbacks) обидва інтерфейси викликають еквівалентне зростання хвостових затримок (P_{99}) у сусідніх сокетних потоках, якщо кількість задач перевищує доступний паралелізм носіїв. Застосування семафорного обмеження (патерн Bulkhead) на рівні прикладного шару наближає затримки сокетних операцій до рівня ізольованого виконання при значно меншому споживанні пам'яті, ніж збільшення пулу носіїв.

Мета дослідження: Експериментальне дослідження меж деградації продуктивності планувальника віртуальних потоків при виконанні нативних операцій через JNI та FFM, а також розробка й оцінка практичних патернів ізоляції ресурсів для запобігання блокуванню спільного пулу потоків-носіїв.

Основні завдання:

1. Розробити автоматизований тестовий стенд на Java та C, що моделює виклики JNI, FFM, callbacks, а також сокетні й файлові операції введення-виведення.

2. Провести багатофакторний експеримент за матрицею параметрів: версії JVM (JDK 21 LTS проти JDK 25 LTS), розмір пулу носіїв, рівень конкурентності та тривалість блокування нативним кодом.

3. Кількісно оцінити взаємний вплив нативного блокування на зростання затримок сусідніх сокетних віртуальних потоків на спільному пулі носіїв.

4. Реалізувати та експериментально порівняти патерни ізоляції (семафорні перегородки Bulkhead, виділені пули платформних потоків) за показниками затримок і витрат пам'яті, сформувавши практичні рекомендації для розробників.

Наукова новизна: Сформовано комплексну порівняльну характеристику взаємного впливу JNI, FFM, нативних callbacks та сокетного/файлового I/O в середовищах JDK 21 та JDK 25, поєднану з кількісною оцінкою ефективності архітектурних патернів ізоляції нативного бар'єру за фіксованого бюджету пам'яті та процесорних ядер.

Практичне значення: Створено модульний стенд для тестування продуктивності віртуальних потоків, набір програмних шаблонів ізоляції ресурсів, а також практичні рекомендації щодо вибору між безпосереднім викликом FFM та делегуванням у виділені пули залежно від тривалості операцій.

Методологія та методи дослідження: Експериментальна інженерія програмного забезпечення: контрольоване лабораторне бенчмаркування (JMH для мікротестів, генератор навантаження для змішаних сценаріїв), низькооверхедне профілювання через JFR та `async-profiler`, порівняльний аналіз патернів ізоляції.

Базовий рівень (Baseline) та вибірка даних: Базовий рівень: для оцінки взаємного впливу – ізольоване виконання сокетних операцій без навантаження нативним кодом; для оцінки патернів ізоляції – стандартне неізольоване виконання віртуальних потоків за замовчуванням (`Executors.newVirtualThreadPerTaskExecutor()`). Вибірка даних – часові ряди затримок (HdrHistogram), профілі використання процесора та розподілу пам'яті (Heap/RSS), дампи подій блокування

`jdk.VirtualThreadPinned`.

Метрики оцінки: Пропускна здатність (ops/sec), хвостові затримки (P_{50}, P_{95}, P_{99}), кратність зростання затримки сокетних операцій під впливом нативних викликів, споживання пам'яті (Heap і RSS), частота подій `jdk.VirtualThreadPinned`.

Інструменти та апаратне забезпечення: OpenJDK 21 та 25 (HotSpot JVM), GCC/Clang, Make, JMH, HdrHistogram, async-profiler, Python/Bash для автоматизації. Апаратна платформа: Intel Core i5-10400F (6 ядер / 12 потоків), 32 GB RAM, ОС Linux із фіксацією ядер через `taskset`.

Етичні ризики: Відсутні. Дослідження використовує виключно відкрите програмне забезпечення та синтетичні тестові навантаження.

Загрози валідності: Внутрішня – спотворення результатів через координоване опущення (Coordinated Omission) у генераторі навантаження (мінімізація: використання HdrHistogram із корекцією затримок); конструктивна – вплив системного кешу сторінок ОС на файловий I/O (мінімізація: тестування `tmpfs` проти синхронного запису `O_DIRECT`); зовнішня – специфіка тестування на одній платформі x86_64 без урахування NUMA-факторів багатопроцесорних серверів.

Мінімальне життєздатне дослідження (MVR) та Stretch Goal: MVR – бенчмаркінговий стенд, що охоплює сценарії JNI та FFM, сокетний I/O, тривалість блокування та кількість носіїв на JDK 21 та JDK 25, з оцінкою базового семафорного патерну Bulkhead проти дефолтної поведінки. Stretch Goal – розробка прототипу адаптивного регулятора конкурентності на базі потокової телеметрії JFR (`jdk.jfr.consumer.RecordingStream`), який динамічно підлаштовує розмір семафора залежно від частоти подій пінінгу.

Ключові верифіковані джерела: JEP 491 ([Src-11]), FCoral ([Src-1], DOI: `https://doi.org/10.1145/3847668`), Schiavio et al. ([Src-2], DOI: `https://doi.org/10.1145/3748522.3779882`), Stanojević et al. ([Src-3], DOI: `https://doi.org/10.1145/3838807`), JobRunr ([Src-12]), InfoQ ([Src-10]) та огляд Java 25 ([Src-24], DOI: `https://doi.org/10.34726/hss.2025.102537`).

Доказова база прогалини: Аналітичний огляд [SGP-3] та картка [GEC-3] підтверджують інженерний розрив: звіти вказують на суттєву деградацію

продуктивності при пінінгу ([Src-12]) та релокацію вузьких місць ([Src-10]), а специфікація JEP 491 ([Src-11]) констатує непереборність нативного закріплення; проте в літературі відсутня систематична кількісна оцінка взаємного блокування задач на спільному пулі носіїв та верифіковані компроміси між патернами bulkheading і споживанням пам'яті.

Критерій фальсифікованості: Публікація відкритого бенчмаркінгового дослідження з аналогічним охопленням факторів (FFM, JNI, callbacks, сокети, JDK 21/25) та оцінкою патернів ізоляції до завершення роботи; або доведення того, що накладні витрати будь-якого прикладного патерну ізоляції повністю нівелюють переваги віртуальних потоків.

Обґрунтування вибору та ризики теми: Висока практична цінність для індустрії (спеціальність 121 «Інженерія програмного забезпечення»), створення конкретних артефактів (тестовий стенд, шаблони патернів, рекомендації), реалістичність виконання на наявному комп'ютері. Головний ризик: більш прикладний інженерний фокус із меншою теоретичною новизною порівняно з аналітичним моделюванням черг; потреба в контролі тривалості багатофакторних прогонів для запобігання перегріву обладнання.

1.4.3 Тема C (міждисплінарна)

Тема C є міждисциплінарним frontier-кандидатом дослідження, що розташовується на перетині системного інжинірингу рантаймів, низькорівневої діагностики ядра операційної системи та прикладного аналізу даних. Вона присвячена подоланню розриву спостережуваності між простором користувача JVM та ядром Linux за допомогою синхронізованої телеметрії (JFR + eBPF/perf), а також побудові швидких сурогатних моделей машинного навчання для передбачення нелінійних точок колапсу продуктивності віртуальних потоків на багатовимірному просторі параметрів.

Заголовок (UA): Міжрівнева діагностика та сурогатне моделювання станів блокування віртуальних потоків на межі JVM та ядра ОС при виконанні нативних викликів FFM і JNI.

Заголовок (EN): Cross-Layer Observability and Surrogate Performance Modeling of Virtual Thread Pinning Across the JVM and OS Kernel Boundary in FFM and JNI Workloads.

Дослідницьке поле: Комп'ютерні науки / Інформаційні системи та технології (спеціальності 122 / 126), Data Science & Systems Observability, низькорівнева діагностика ядра ОС (Linux kernel & eBPF tracing), сурогатне машинне навчання у системному аналізі продуктивності.

Наукова проблема: Пінінг віртуальних потоків на межі FFM/JNI є складним міжрівневим феноменом: планувальник JVM керує відображенням віртуальних потоків на потоки-носії у просторі користувача (user space), тоді як нативні виклики (downcalls), зворотні виклики (callbacks) та змішані файлові/мережеві операції переходять у некеровані C-фрейми та системні виклики ядра ОС (`futex`, `epoll_wait`, `read`/`write`, `nanosleep`). Аналіз цієї межі виключно через метрики JVM або виключно через метрики ОС приховує механіку взаємодії нативних фреймів із планувальником ядра в умовах насичення пулу носіїв. Крім того, повне вичерпне бенчмаркування 8-вимірного комбінаторного простору конфігурацій є обчислювально затратним.

Доказова база проблеми:

1. Нормативна специфікація OpenJDK JEP 491 ([Src-11]): офіційно підтверджує, що ліквідація моніторного пінінгу в JDK 24 не поширюється на нативний код: виклики через JNI та Panama FFM при переході через системний бар'єр C ABI у ядро ОС примусово утримують платформні потоки-носії.

2. Обмеження існуючих інструментів тестування конкурентності JVM (Fray, ACM OOPSLA 2025, [Src-6], DOI: `https://doi.org/10.1145/3764119`): сучасні платформи детермінованого тестування багатопоточності HotSpot діють виключно на рівні байткоду/JVMTI через shadow locks і принципово не відстежують стани перемикання контексту ядра Linux (`sched_switch`, черги очікування на `futex`), що приховує причини міжрівневих блокувань.

3. Нестаціонарність та фазові переходи рантайму JVM (Antoch et al., ACM OOPSLA 2026, [Src-4], DOI: `https://doi.org/10.1145/3839448`): доведено, що сучасні навантаження на JVM не досягають стаціонарного стану через фонову деоптимізацію та перебудову JIT-профілів, що зумовлює нелінійну природу деградації та вимагає використання нелінійних моделей машинного навчання (GBDT).

4. Емпіричні свідчення виснаження носіїв у промислових бенчмарках (JobRunr, [Src-12]; InfoQ, [Src-10]): JobRunr зафіксував падіння throughput у 14 разів

(з 462 до 33 jobs/s) та зліт хвостових затримок P_{99} у 27 разів (з 253 мс до 6799 мс) під час пінінгу носія; InfoQ довів явище зміщення вузького місця (bottleneck relocation) та виникнення $\sim$ 37% помилок при неізольованому масштабуванні віртуальних потоків.

5. Обмеження сучасного міжрівневого eBPF-трасування JVM (Liegsalz & Brunnert, SSP 2026, [Src-25]): у дослідженні трасування Java через Linux eBPF автори визнають, що їхня модель прив'язана до задач ядра (`task_struct`) і повністю втрачає кореляцію при переході на віртуальні потоки; крім того, трасування через DTrace USDT-проби збільшує затримку у 6.1 раза (+507%) і спотворює CPU time у 7.4 раза через оверхед пасток переходу в ядро, що обґрунтовує потребу у низькооверхедному конвеєрі JFR + eBPF kernel tracepoints.

Дослідницька прогалина: Блокування віртуальних потоків на нативному коді створює міжрівневу «сліпу зону». Профайлери Java не бачать, що відбувається всередині ядра Linux (блокування системних викликів і м'ютексів), а системні утиліти ядра (eBPF) бачать лише процесор і втрачають контекст віртуальних потоків. При цьому спроби детального системного трасування сповільнюють програму на 500% (ефект спостерігача), а перевірити всі комбінації налаштувань вручну – нереально довго.

• **Міжрівнева телеметрія (Java + Linux):** як поєднати легкі логи Java (JFR) із лічильниками ядра Linux без падіння швидкодії, щоб точно діагностувати причину зависання потоків у системних викликах ядра.

• **ML-прогнозування збоїв (сурогатна модель):** як за допомогою машинного навчання на основі невеликої вибірки тестів заздалегідь передбачати момент колапсу затримок (P_{99}) для невідомих налаштувань сервера без тривалого вичерпного тестування.

Тип прогалини: Methodological gap.

Об'єкт дослідження: Міжрівневий життєвий цикл виконання гібридних Java/нативних навантажень, що охоплює планувальник віртуальних потоків JVM, інтерфейси нативної взаємодії JNI/FFM та планувальник задач ядра операційної системи Linux.

Предмет дослідження: Синхронізована міжрівнева телеметрія (поєднання подій JVM Flight Recorder із лічильниками ядра Linux `perf`/eBPF) та статистичні сурогатні моделі машинного навчання (Response Surface Methodology, GBDT

– LightGBM/XGBoost, SHAP-аналіз взаємодій факторів) для виявлення та прогнозування колапсу продуктивності (хвостових затримок P_{99}).

Дослідницькі питання (RQ):

• **RQ1:** Які характерні сигнатури планування в ядрі ОС (частота перемикання контексту `sched_switch`, частка перебування в режимі ядра, блокування на `futex`, час очікування в черзі планувальника) розрізняють пінінг нативних фреймів JNI та FFM від кооперативного зняття віртуальних потоків (unmounting) та блокувань сокетного/файлового I/O в різних версіях JVM?

• **RQ2:** Як зворотні нативні виклики (Java $\rightarrow$ C $\rightarrow$ блокуючий Java-код) проявляються у комбінованій телеметрії JVM та ядра ОС у версіях, де усунуто пінінг на моніторах `synchronized` (JDK 24+ / JEP 491), порівняно з легасі-версіями (JDK 21)?

• **RQ3:** Чи здатна інтерпретована сурогатна модель машинного навчання, навчена на вибірці простору параметрів за методом латинського гіперкуба (Latin Hypercube Sampling / дробовий факторний план), з високою точністю ($R^2 > 0.85$, $F_1 > 0.9$) прогнозувати настання нелінійного колапсу затримок P_{99} та голодування пулу носіїв на невідомих конфігураціях?

Наукова гіпотеза: Поєднання низькорівневих метрик стану потоків ядра ОС (tracepoints ядра, апаратні лічильники `perf`) із JFR-подіями пінінгу JVM дозволяє ідентифікувати межі експлуатаційних режимів (наприклад, приховану конкуренцію за нативні м'ютекси ядра проти кооперативного розвантаження через `epoll`), які неможливо розрізнити лише за метриками простору користувача JVM; крім того, ансамблеві деревні сурогатні моделі (LightGBM/XGBoost із SHAP-атрибуцією) здатні передбачати хвостову деградацію P_{99} та виявляти домінантні міжфакторні взаємодії (зокрема 3-факторну взаємодію «наявність callbacks $\times$ тривалість блокування $\times$ співвідношення конкурентності до розміру пулу носіїв») на основі дробової вибірки ($< 20\%$ від повного комбінаторного простору).

Мета дослідження: Побудова синхронізованого програмного конвеєра міжрівневої телеметрії (JVM-to-OS) та навчання інтерпретованої сурогатної моделі для діагностики, аналізу факторної чутливості та швидкого прогнозування нелінійного колапсу продуктивності віртуальних потоків на межі FFM/JNI під багатовимірним навантаженням.

Основні завдання:

1. Розробити синхронізований вимірювальний комплекс, що зіставляє події JVM JFR (`jdk.VirtualThreadPinned`, `jdk.ThreadPark`) із точками трасування ядра Linux (`sched:sched_switch`, системними викликами `syscalls:sys_enter_*`, лічильниками `perf_event_open/bpftrace`) для JNI, FFM, callbacks, сокетів, файлів та сліпу.

2. Спроєктувати та реалізувати план багатофакторного експерименту (поєднання повного факторного ядра з вибіркою за латинським гіперкубом LHS для неперервних параметрів) у розрізі версій JDK 21 та JDK 25 за фіксованого бюджету ядер та пам'яті.

3. Провести дисперсійний та причинно-наслідковий аналіз чутливості (ANOVA, індекси Соболя, значення SHAP) для кількісного ранжування внеску кожного з 8 факторів та їхніх взаємодій у деградацію квантилів затримок.

4. Навчити, валідувати та порівняти сурогатні моделі (поліноміальні моделі поверхні відгуку RSM та градієнтний бустинг LightGBM/XGBoost) для прогнозування пропускної здатності та кліфів P_{99} на відкладених (out-of-distribution) тестових конфігураціях.

5. Розробити прототип легкої утиліти автоматизованої експрес-діагностики («Loom Cross-Layer Advisor»), яка аналізує короткий комбінований профіль та оцінює дистанцію до режиму голодування носіїв.

Наукова новизна: Міждисциплінарний синтез методів системного інжинірингу рантаймів, низькорівневого трасування ядра Linux та сурогатного машинного навчання, що вперше забезпечує кількісну оцінку міжфакторної взаємодії 8 параметрів на межі JNI/FFM та розкриває механізми поведінки ядра ОС при блокуванні віртуальних потоків на спільних потоках-носіях.

Практичне значення:

• Відкритий інструментальний скрипт міжрівневого моніторингу (eBPF + JFR) для низькооверхедної діагностики виробничих затримок у Linux-середовищах.

• Опублікований відкритий табличний датасет багатофакторних прогонів для спільноти дослідників продуктивності JVM.

• Легковажна сурогатна модель та утиліта експрес-оцінки конфігурації для автоматизованого тюнінгу параметрів пулу носіїв у CI/CD або рантаймі без

проведення тривалого вичерпного тестування.

Методологія та методи дослідження: Теорія планування експерименту (Design of Experiments – DoE: Full Factorial + Latin Hypercube Sampling); системне низькорівневе трасування ядра Linux (`perf`, `bpftrace`, kernel tracepoints); дисперсійний аналіз чутливості (індекси Соболя, ANOVA); машинне навчання з учителем (Response Surface Methodology, GBDT: LightGBM / XGBoost) із k-кратною крос-валідацією та інтерпретацією через адитивні пояснення Шеплі (SHAP values).

Базовий рівень (Baseline) та вибірка даних: Однорівневі регресійні евристики та лінійні моделі виключно на базі метрик простору користувача JVM (без системних лічильників ядра), а також емпіричні базові профілі JDK 21 для JNI/FFM без розділення режимів блокування ядра та кооперативного перемикання. Вибірка даних – публічний багатовимірний табличний датасет експериментальних запусків (кілька тисяч конфігураційних точок), де кожен запис містить 8 незалежних змінних (факторів), набір проміжних міжрівневих метрик (JFR-події, лічильники ядра ОС, перемикання контексту) та цільові показники (throughput, квантилі затримок P_{50}–$P_{99.9}$, бінарна мітка starvation cliff).

Метрики оцінки: Якість сурогатних моделей: коефіцієнт детермінації (R^2), RMSE, MAPE для логарифма затримки $\ln(P_{99})$, Precision, Recall та F_1-score для класифікації настання кліфу голодування. Факторна чутливість: індекси Соболя першого та повного порядку, абсолютні значення SHAP. Системні міжрівневі метрики: швидкість добровільних/примусових перемикань контексту (`voluntary_ctxt_switches`, `nonvoluntary_ctxt_switches` / sec), співвідношення Kernel/User CPU time, середня тривалість блокування на `futex`, частота подій `jdk.VirtualThreadPinned`.

Інструменти та апаратне забезпечення: OpenJDK 21 та 24/25 (HotSpot JVM), GCC/Clang, Make, Linux `perf`, `bpftrace`, BCC / eBPF kernel tracepoints, Python 3.11+ (Pandas, SciPy, Scikit-learn, XGBoost, LightGBM, SHAP, SALib, Matplotlib, Seaborn), JMH та Java Flight Recorder (JFR). Апаратна платформа: Intel Core i5-10400F (6 ядер / 12 потоків), 32 GB RAM, ОС Linux. Навчання табличних ML-моделей на вибірках у кілька тисяч рядків займає менше 60 секунд на 6 ядрах.

Етичні ризики: Відсутні. Дослідження спирається на синтетичні навантаження, відкриті системні інтерфейси та ліцензійно чисті компоненти без персональних даних або комерційної таємниці.

Загрози валідності: Внутрішня (Observer effect) – накладні витрати одночасного трасування JFR та ядра Linux можуть спотворити вимірювання (мінімізація: внутрішньоядерні eBPF map-агрегації та лічильники `perf stat` замість дискового `perf stat` дампу кожного виклику з фіксацією оверхеду $< 2\%$); конструктивна – перенавчання сурогатної моделі (мінімізація: тестування на відкладених конфігураціях з інтерполяцією та екстраполяцією); зовнішня – специфіка ядра Linux та планувальника CFS/EEVDF (мінімізація: фіксація версії ядра Linux 6.8+ та конфігурації у вихідних даних).

Мінімальне життєздатне дослідження (MVR) та Stretch Goal: MVR – повний емпіричний датасет за 8 факторами, зіставлений із лічильниками `perf stat` та ключовими подіями ядра, розрахунок глобальної чутливості факторів через SHAP, та валідована сурогатна модель (LightGBM/XGBoost), що передбачає настання P_{99} starvation cliff із точністю $F_1 > 0.85$. Stretch Goal – розробка компактної консольної утиліти («Loom Cross-Layer Advisor»), яка знімає короткий 10-секундний комбінований профіль (JFR + `perf`) працюючого застосунку і на основі навченої сурогатної моделі сигналізує, якщо поточна інтенсивність нативних викликів перебуває ближче ніж за 15% до точки колапсу продуктивності carrier-пулу.

Ключові верифіковані джерела: JEP 491 ([Src-11]), FCoral ([Src-1], DOI: `https://doi.org/10.1145/3847668`), Antoch et al. ([Src-4], DOI: `https://doi.org/10.1145/3839448`), Fray ([Src-6], DOI: `https://doi.org/10.1145/3764119`), Schiavio et al. ([Src-2], DOI: `https://doi.org/10.1145/3748522.3779882`), InfoQ ([Src-10]), JobRunr ([Src-12]) та дослідження eBPF jAgent Liegsalz & Brunnert ([Src-25]).

Доказова база прогалини: Аналітичні огляди [SGP-3] та картка [GEC-3] фіксують гострий міжрівневий розрив спостережуваності: провідні платформи тестування байткоду ([Src-6]) сліпі до станів ядра ОС, тоді як новітні eBPF-конвеєри ([Src-25]) офіційно визнають втрату кореляції з віртуальними потоками та колосальний оверхед перехоплення (+507%), підтверджуючи відсутність легковажних міжрівневих методів предиктивної діагностики голодування носіїв.

Критерій фальсифікованості: Якщо експериментально виявиться, що багатофакторні взаємодії між JNI/FFM, callbacks, сокетами та розміром пулу

носіїв є абсолютно лінійними/адитивними і не мають нелінійних точок перегину (cliffs), що робить застосування нелінійних сурогатних моделей надлишковим; або якщо до моменту захисту буде опубліковано аналогічне міжрівневе дослідження з комбінованим eBPF+JFR аналізом та сурогатним моделюванням межі віртуальних потоків для FFM/JNI.

Обґрунтування вибору та ризики теми: Ідеальний міждисциплінарний науковий профіль на стику системного програмування та аналізу даних (спеціальності 122 «Комп'ютерні науки» або 126 «Інформаційні системи та технології»). Забезпечує високу наукову новизну (публікабельність у Scopus/WoS), відкритий цінний датасет і прикладну утиліту, демонструючи володіння як низькорівневим системним стеком Linux, так і сучасними методами Data Science / Machine Learning. Головний ризик полягає у вимозі до подвійної експертизи (double skill curve): дослідник повинен одночасно володіти низькорівневим інструментарієм ядра Linux (C, `perf`, eBPF/`bpftrace`) та стеком машинного навчання (Python, Scikit-learn, XGBoost, SHAP, Sobol analysis), що вимагає суворого контролю часових витрат.

1.5 Порівняння тем

Зведене порівняння трьох кандидатів магістерських тем дослідження (A, B та C) за ключовими класифікаційними та методологічними критеріями наведено в табл. 1.2.

Таблиця 1.2 – Порівняльна таблиця тем A, B і C

Критерій	Тема A	Тема B	Тема C
Тип теми	Аналітична / теоретична	Прикладна / інженерна	Міждисциплінарна / frontier
Тип gap	Theoretical gap	Empirical gap	Methodological gap

Продовження таблиці 1.2

Критерій	Тема A	Тема B	Тема C
Метод	Моделювання черг (теорія масового обслуговування), чисельний аналіз (Python), бенчмаркінг на базі JMH	Експериментальне бенчмаркування (JMH, генератор навантаження), профілювання (JFR, `async-profiler`), патерни bulkheading	Планування експерименту (DoE, LHS), системне трасування ядра Linux (eBPF, `perf`), аналіз чутливості (Sobol, SHAP), GBDT (LightGBM/ XGBoost)
Головна метрика	Похибка MAPE, R^2 апроксимації черги, хвостові затримки (P_{95}, P_{99})	Пропускна здатність (ops/s), затримка P_{99}, кратність зростання затримок сокетів, пам'ять (Heap/RSS)	Метрики ML (R^2, RMSE, F_1-score для starvation cliff), значення SHAP, перемикання контексту (`sched_switch`)
Головний результат	Розрахункова модель змішаної черги, оцінка порогу навантаження пулу носіїв, верифікаційні дані JNI та FFM	Модульний бенчмаркінговий стенд, оцінка взаємного впливу навантажень, набір патернів ізоляції, інженерні рекомендації	Синхронізований конвеєр телеметрії (JFR + eBPF/`perf`), публічний датасет, сурогатна модель прогнозування P_{99}, утиліта «Loom Cross-Layer Advisor»

Продовження таблиці 1.2

Критерій	Тема A	Тема B	Тема C
Складність стенда	Низька (локальний стенд JMH/JFR для емпіричної валідації моделі черги)	Середня (модульний автоматизований стенд Java/C зі змішаним навантаженням та реалізацією патернів bulkheading)	Висока (міжрівневий конвеєр трасування ядра Linux через eBPF/`perf` + HotSpot JFR із мінімізацією оверхеду $< 2\%$ та ML-пайплайн)
Головний ризик	Складність аналітичного опису неекспоненційних нативних операцій (перехід до наближених чисельних методів)	Менша теоретична новизна (більш прикладний фокус); ризик перегріву та троттлінгу CPU через тривалість багатофакторних запусків	Вимога подвійної експертизи (double skill curve: Linux kernel/eBPF та Data Science/ML); ризик розмивання фокусу між трасуванням і моделюванням

1.6 Peer review

1.6.1 Відгуки щодо теми A

1.6.2 Відгуки щодо теми B

1.6.3 Відгуки щодо теми C

1.7 Decision Matrix

ВИСНОВОК